AnvilCLI — Technical Documentation

> ***Version**: 0.2.0 **Author**: Oscar Canton Garcia **Platform**: macOS (darwin arm64/amd64) **Language**: Go 1.23+*
> ***Repository**: github.com/magnoscg/Ios-Architecture*

Table of Contents

Overview

AnvilCLI is a Go-based command-line tool that scaffolds production-ready iOS projects following **Clean Architecture + MVVM + Router** patterns. It operates in two modes:

- **Project Mode** — Generates a full iOS project: Xcode project, all architecture layers (Domain, Data, Features), Git initialization, and optional AI coding packs.
- **Tools Mode** — Installs only AI coding packs (docs, skills, commands, agents) into an existing project.

The tool provides an interactive terminal UI (TUI) built with BubbleTea v2 and Lipgloss, featuring a Catppuccin Mocha color theme.

Installation & Build

From Source

```
# Build
make build                    # → bin/anvil

# Install to GOPATH
make install

# Run
anvil init                    # Interactive TUI wizard
anvil feature <name>          # Add feature to existing project
anvil version                 # Print version
```

From GitHub Release

```
# Download latest release
curl -L https://github.com/magnoscg/Ios-
Architecture/releases/latest/download/anvil_<version>_darwin_arm64.tar.gz | tar xz
./anvil init
```

Makefile Targets

Target	Description
`make build`	Build binary with version ldflags to `bin/anvil`
`make test`	Run all unit tests
`make test-integration`	Run integration tests (build tag `integration`)
`make install`	Install to GOPATH/bin
`make clean`	Remove bin/ and test cache
`make lint`	Run golangci-lint
`make fmt`	Format all Go files
`make all`	fmt + lint + test + build

Project Structure

```
cmd/anvilcli/                       # CLI entry points
    main.go                         # Program entry
    root.go                         # Root cobra command
    cmd_init.go                     # `anvil init` — assembles dependencies, launches TUI
    feature.go                      # `anvil feature <name>` — scaffolds a feature
    version.go                      # `anvil version` — prints version (ldflags)

internal/config/                    # Configuration & types
    config.go                       # ProjectConfig struct (all project settings)
    defaults.go                     # Default values (iOS 18.0, Swift 6.0, schemes)
    pack.go                         # Pack struct definition
    pack_registry.go                # 6 packs + ResolveDependencies + ValidatePacks
    anvilfile.go                    # .anvil.yml marker (read/write)
    namer.go                        # ToPascalCase, ToCamelCase, ToSnakeCase
    errors.go                       # 9 custom error types

internal/deps/                      # System dependency checker
    checker.go                      # SystemChecker.Check() — 5 dependency checks
    parser.go                       # Version string parsers (Xcode, git, etc.)

internal/feature/                   # Feature scaffolder
    scaffolder.go                   # FeatureScaffolder.Scaffold() — generates ~30 files
    layout.go                       # File job list (template path → destination path)
    namer.go                        # Feature naming helpers

internal/generator/                 # Project generation engine
    generator.go                    # DefaultProjectGenerator — 8-step pipeline
    renderer.go                     # Template engine (text/template + custom FuncMap)
    context.go                      # ProjectTemplateContext for template rendering
```

```
    xcodeproj.go                    # .xcodeproj bundle generator
    xcodeproj_context.go            # PbxprojContext + deterministic UUID generation
    pack_renderer.go                # Pack installation engine (10-step per pack)
    settings_merger.go              # JSON settings merge (.claude/settings.json)
    git.go                          # Git runner (init, add, commit)
    writer.go                       # DiskWriter abstraction
    embed.go                        # go:embed for templates/

internal/tui/                       # Terminal UI (BubbleTea v2 + Lipgloss)
    model.go                        # WizardModel — root model, 4-screen orchestrator
    mode.go                         # Screen 1: Project vs Tools selection
    setup.go                        # Screen 2: Form + Features + Environment + Scope
    pack_picker.go                  # Screen 3: AI pack selector (tools mode only)
    generate.go                     # Screen 4: Progress spinner + completion
    layout.go                       # Brand header, footer, section headers, version
    theme.go                        # Catppuccin Mocha color palette
    keys.go                         # Key press helpers

templates/                          # Embedded templates (go:embed → compiled into binary)
    base/                           # Base project (~60 Swift template files)
    swiftdata/                      # SwiftData persistence templates (conditional)
    feature/                        # Feature scaffolder templates (~30 files)
    example/                        # Example feature templates (conditional)
    ai-packs/                       # 6 AI pack directories

scripts/                            # Build and test helpers
.github/workflows/                  # CI (ci.yml) + Release (release.yml)
.goreleaser.yml                     # goreleaser v2 config
Makefile                            # Build targets
```

CLI Commands

`anvil init`

Interactive TUI wizard that creates a new iOS project or installs AI coding tools.

Dependency injection chain:

1. `deps.NewSystemChecker` — checks system dependencies
2. `generator.NewDiskWriter` — file system writer
3. `generator.NewRenderer` — template engine
4. `generator.NewXcodeProjGenerator` — Xcode project builder
5. `generator.NewGitRunner` — git operations
6. `config.FileMarkerReadWriter` — .anvil.yml handler
7. `feature.NewFeatureScaffolder` — example feature generator
8. `generator.NewPackRenderer` — AI pack installer
9. `generator.NewProjectGenerator` — orchestrates everything

All dependencies are injected into `tui.NewWizardModel()` which runs the BubbleTea program.

`anvil feature <name>`

Adds a new feature to an existing AnvilCLI-generated project.

Flow:

1. Finds project root via `.anvil.yml` marker (walks up directory tree)

2. Interactive form (charmbracelet/huh library):
    - Include local data source? (yes/no)
    - Include Keychain data source? (yes/no)
    - Include RouteResolver? (yes/no)
3. Calls `FeatureScaffolder.Scaffold()` → generates ~30 files
4. Prints created files and wiring instructions

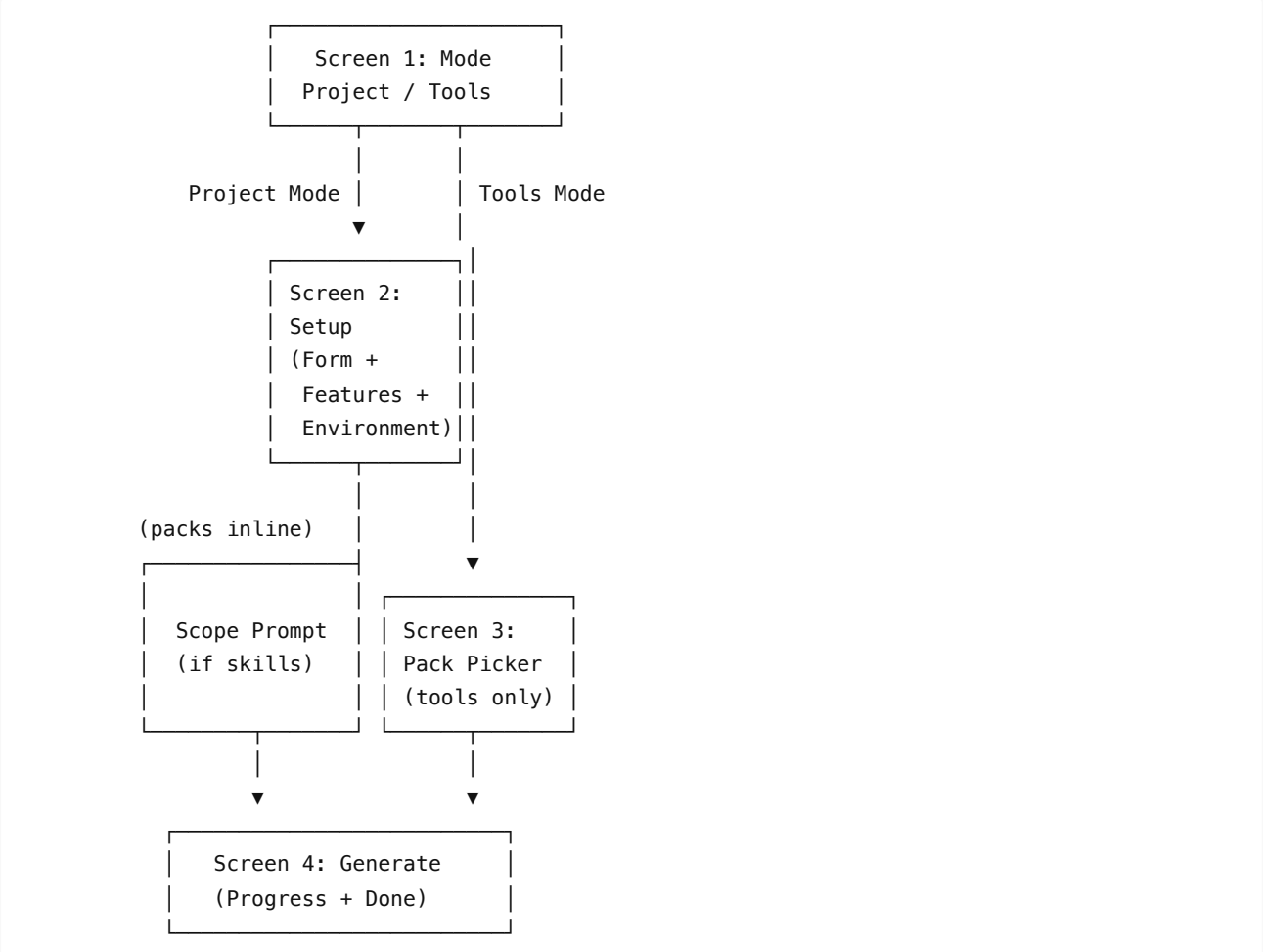
`anvil version`

Prints the CLI version. Version is set at build time via ldflags:

```
go build -ldflags "-X main.Version=0.2.0" ./cmd/anvilcli
```

TUI Wizard — 4 Screens

Screen Flow



Screen 1: Mode Selection (`mode.go`)

Two radio options:

- **Create new iOS project** — Full scaffold with Xcode project, architecture, AI tools
- **Install AI coding tools** — Add docs, skills, commands to existing project

Controls: up/down navigate, enter select, q quit

Screen 2: Setup (`setup.go`)

5 Text Fields:

Field	Default	Validation
Project Name	*(required)*	`^[A-Za-z][A-Za-z0-9_-]*$`
Bundle ID	Auto-generated from name	Free text
iOS Version	`18.0`	Free text
Swift Version	`6.0`	Free text
Schemes	`Dev, Stg, Production`	Comma-separated

Hierarchical Features Section:

```
◆ Features
  [ ] SwiftData persistence
  [ ] Example feature
  ▸ AI Coding Packs              ← collapsed by default
      [ ] iOS Architecture
      [ ] PRD Planner
      [ ] Axiom iOS
      [ ] Swift Design Patterns
      [ ] Gitflow
      [ ] GitHub Actions
```

- Groups expand/collapse with right/left arrows or space
- Pack sub-items toggle with space
- Dependency resolution: selecting PRD Planner auto-selects iOS Architecture

Environment Check (async background task):

Dependency	Required	Detection Command
Xcode	Yes	`xcodebuild -version`
git	Yes	`git --version`
claude-code	No	`claude --version`
swiftlint	No	`swiftlint version`
swiftformat	No	`swiftformat --version`

Scope Prompt (overlay, shown if any selected pack has `HasSkills=true`):

- Project scope → `.claude/skills/`
- Global scope → `~/.claude/skills/`

Controls: up/down navigate, tab cycle, space toggle, left/right expand/collapse, enter confirm, esc back

Screen 3: Pack Picker (`pack_picker.go`)

Used only in **Tools Mode**. Lists all 6 packs with:

- Manual selection (green checkbox)
- Auto-selection (muted checkbox + "(auto)" badge) for transitive dependencies
- Status messages ("Required by X — deselect it first")

- Pack descriptions shown on focus

Dependency resolution:

- `config.ResolveDependencies()` computes transitive closure
- Cannot deselect a pack that is required by another selected pack
- Deselecting a pack recalculates all auto-selections

Controls: up/down or j/k navigate, space toggle, enter confirm, esc back

Screen 4: Generate (`generate.go`)

Progress Display:

- Animated spinner (10 frames: ⠋⠙⠹⠸⠼⠴⠦⠧⠇⠏)
- Step list with status icons: ✓ done, ⠋ active, ○ pending, ✗ failed

Project Mode Steps:

1. Creating project directory
2. Rendering templates
3. Generating Xcode project
4. Initializing git repository
5. Writing project config

Tools Mode Steps:

1. Resolving dependencies
2. Installing tools
3. Writing config

Completion Screen:

- Success box with timing, file count, directory path
- Next steps instructions
- Press enter or q to exit

Configuration System

ProjectConfig (`internal/config/config.go`)

```
type ProjectConfig struct {
    Name             string        // PascalCase project name
    BundleID         string        // iOS bundle identifier
    Organization     string        // Organization (derived from BundleID)
    IOSVersion       string        // Minimum iOS deployment target
    SwiftVersion     string        // Swift language version
    Schemes          []string      // Build scheme names
    OutputDir        string        // Target directory
    IncludeSwiftData bool          // Enable SwiftData persistence
    IncludeExample   bool          // Generate example feature
    AIPacks          []string      // Selected AI pack slugs
    SkillsScope      string        // "project" or "global"
    Mode             ProjectMode   // ModeProject or ModeTools
}
```

Defaults (`internal/config/defaults.go`)

Setting	Default
iOS Version	`18.0`
Swift Version	`6.0`
Schemes	`["Dev", "Stg", "Production"]`
IncludeSwiftData	`false`
IncludeExample	`false`
SkillsScope	`"project"`

AnvilMarker — `.anvil.yml` (`internal/config/anvilfile.go`)

Written to the project root after generation. Used by `anvil feature` to locate project context.

```yaml
version: "0.2.0"
project_name: MyApp
bundle_id: com.company.MyApp
ios_version: "18.0"
swift_version: "6.0"
schemes:
  - Dev
  - Stg
  - Production
ai_packs:
  - ios-architecture
  - prd-planner
skills_scope: project
created_at: 2025-03-20T12:34:56Z
```

Legacy field `include_claude: true` is auto-migrated to `ai_packs: ["ios-architecture"]`.

AI Coding Packs

Pack Registry (`internal/config/pack_registry.go`)

#	Slug	Display Name	Requires	HasSkills	Description
1	`ios-architecture`	iOS Architecture	—	No	Clean Architecture rules, anti-patterns, 13 reference docs
2	`prd-planner`	PRD Planner	`ios-architecture`	No	Dev workflow commands, agents, 683 Kavsoft tutorials
3	`axiom-ios`	Axiom iOS	—	No	Simulator skills, audits, debugging agents
4	`swift-design-patterns`	Swift Design Patterns	—	**Yes**	22 pattern skills + 3 overview skills
5	`gitflow`	Gitflow	—	**Yes**	Git workflow skill (branching, commits, PRs)
6	`github-actions`	GitHub Actions	—	No	CI + Release workflow templates

Dependency Resolution

`ResolveDependencies(selected []string) []string` computes the transitive closure:

1. Start with user-selected slugs
2. For each, add its `Requires` dependencies
3. Repeat until stable
4. Return in topological order (dependencies first, stable by display order)

Pack Installation Pipeline (`internal/generator/pack_renderer.go`)

For each selected pack, the PackRenderer executes up to 10 steps:

Step	Source	Destination
1. CLAUDE.md	`CLAUDE.md.tmpl` + `CLAUDE-section.md.tmpl`	`<project>/CLAUDE.md` (composite)
2. Docs	`docs/`	`<project>/.claude/docs/`
3. Commands	`commands/`	`<project>/.claude/commands/`
4. Agents	`agents/`	`<project>/.claude/agents/`
5. Dev config	`dev/`	`<project>/.dev/` (rendered)
6. Plan templates	`plan/`	`<project>/plan/` (rendered)
7. Skills	`skills/`	`.claude/skills/` or `~/.claude/skills/`
8. Tutorials	`tutorials/`	`~/.claude/tutorials/` (always global)
9. Settings merge	`settings-merge.json`	`.claude/settings.json` (JSON merge)
10. Workflows	`workflows/`	`.github/workflows/` (rendered)

Pack Contents Detail

ios-architecture:

- `CLAUDE.md.tmpl` — Full project CLAUDE.md with architecture rules, anti-patterns, workflow
- `docs/` — 13 reference documents:
  - `ARCHITECTURE.md` , `PROJECT-STRUCTURE.md` , `new-feature.md`
  - `swiftui-code-style.md` , `design-system.md`
  - `swift-concurrency.md` , `performance.md`
  - `swiftdata.md` , `networking.md`
  - `security-privacy.md` , `diagnostics.md`
  - `testing.md` , `create-tests.md` , `workflow-qa.md`

prd-planner:

- `CLAUDE-section.md.tmpl` — Appends workflow section to CLAUDE.md
- `commands/` — Dev workflow commands (`dev-prd` , `dev-plan` , `dev-build` , `dev-status` , etc.)
- `agents/` — Specialized agents (DEV-SPEC-WRITER, DEV-TASK-PLANNER, DEV-IMPLEMENTER, etc.)
- `dev/` — `arch-index.md` , `skill-registry.md` , `ast-patterns.yml`
- `plan/` — `INDEX.md` template
- `settings-merge.json` — Merges agent/command config into settings
- `tutorials/` — 683 Kavsoft tutorials index

axiom-ios:

- `CLAUDE-section.md.tmpl` — Axiom skills reference, simulator commands

- `settings-merge.json` — Axiom plugin configuration

swift-design-patterns:

- `skills/` — 25 skill files (22 patterns + 3 overviews: creational, structural, behavioral)

gitflow:

- `skills/` — `git-workflow-skill.md`
- `CLAUDE-section.md.tmpl` — Git conventions

github-actions:

- `workflows/` — `ci.yml.tmpl`, `release.yml.tmpl`

Generator Pipeline

Project Mode — 8-Step Pipeline (`internal/generator/generator.go`)

```
Generate(ctx, cfg) → GenerationResult
```

Step	Action	Rollback on Failure
1	Create project directory (`OutputDir/ProjectName/`)	Yes
2	Render base templates (App, Core, Domain ~60 Swift files)	Yes
3	Render scheme xcconfigs (one per scheme)	Yes
4	Render SwiftData templates (if `IncludeSwiftData`)	Yes
5	Render AI Packs (if any selected)	Yes
6	Scaffold Example feature (if `IncludeExample`)	Yes
7	Generate .xcodeproj bundle (pbxproj, workspace, schemes)	No (non-fatal)
8	Git init + add + commit	No (non-fatal)
9	Write .anvil.yml marker	—

Rollback: If steps 1-6 fail, the entire project directory is removed.

Tools Mode (`GenerateToolsOnly`)

- Skips directory creation, base templates, xcodeproj, git
- Only runs PackRenderer for selected packs into `OutputDir`

Generation Result

```
type GenerationResult struct {
    ProjectDir         string
    FilesCreated       []string        // Relative paths
    XcodeProjectOutput string          // Summary
    GitOutput          string          // Result or error
    Duration           time.Duration
}
```

Feature Scaffolder

Usage

```
anvil feature PokemonDetail
```

Generated Files (~30 per feature)

```
Domain/<Feature>/
    Models/<Feature>Model.swift
    UseCases/<Feature>UseCase.swift
    UseCases/<Feature>UseCaseImpl.swift

Data/<Feature>/
    DTO/<Feature>DTO.swift
    Mappers/<Feature>DTOMapper.swift
    DataSources/<Feature>RemoteDataSource.swift
    DataSources/<Feature>LocalDataSource.swift          ← conditional
    DataSources/<Feature>KeychainDataSource.swift       ← conditional
    Repositories/<Feature>Repository.swift
    Repositories/<Feature>RepositoryImpl.swift

Features/<Feature>/
    DI/<Feature>Factory.swift
    UI/<Feature>View.swift
    UI/<Feature>State.swift
    UI/<Feature>Decorator.swift
    Presentation/ViewModels/<Feature>ViewModel.swift
    Presentation/Mappers/<Feature>DecoratorMapper.swift
    Navigation/<Feature>Router.swift
    Navigation/<Feature>RouteResolver.swift              ← conditional

Tests/Domain/<Feature>/
    <Feature>UseCaseTests.swift

Tests/Data/<Feature>/
    <Feature>RepositoryTests.swift
    <Feature>DTOMapperTests.swift

Tests/Features/<Feature>/
    <Feature>ViewModelTests.swift
    <Feature>DecoratorMapperTests.swift

Tests/Mocks/
    <Feature>UseCaseMock.swift
    <Feature>RepositoryMock.swift
    <Feature>RouterMock.swift
    <Feature>RemoteDataSourceMock.swift
    <Feature>DTOMapperMock.swift
    <Feature>DecoratorMapperMock.swift
```

Conditional Files

Flag	File Generated

`IncludeLocalDataSource`	`<Feature>LocalDataSource.swift`
`IncludeKeychain`	`<Feature>KeychainDataSource.swift`
`IncludeRouteResolver`	`<Feature>RouteResolver.swift`

Wiring Instructions

After scaffolding, the CLI prints manual integration steps:

- Register `<Feature>Router` in `AppRouter`
- Register `<Feature>RouteResolver` in app root (if applicable)
- Wire `<Feature>Factory` into `AppDependencies`

Xcode Project Generation

.xcodeproj Bundle Structure

```
<ProjectName>.xcodeproj/
    project.pbxproj                         ← main project file
    project.xcworkspace/
        contents.xcworkspacedata            ← workspace reference
    xcshareddata/
        xcschemes/
            <ProjectName>-Dev.xcscheme      ← one per scheme
            <ProjectName>-Stg.xcscheme
            <ProjectName>-Production.xcscheme
```

Deterministic UUID Generation

All UUIDs in the pbxproj are deterministic using FNV1a hashing:

- Input: `projectName + uuidSeedString`
- Same project name always produces identical UUIDs
- Enables reproducible builds

UUID Set (~30 named UUIDs)

UUID	Purpose
`RootGroup`	Top-level PBXGroup
`AppTarget`	Application target
`TestTarget`	Unit test target
`AppBuildConfigDebug/Release`	Build configurations
`SwiftDataFramework`	SwiftData framework reference (conditional)
`SwiftDataBuildFile`	SwiftData build file (conditional)
Per-scheme UUIDs	Debug/Release configs for each scheme

pbxproj Template Context

```
type PbxprojContext struct {
    ProjectName      string
```

```
    BundleID         string
    Organization     string
    IOSVersion       string
    SwiftVersion     string
    TestTargetName   string
    Schemes          []SchemeContext
    UUIDs            UUIDSet
    IncludeSwiftData bool
}
```

Template System

Embedded FS (`go:embed`)

```
//go:embed base swiftdata feature example ai-packs
var TemplateFS embed.FS
```

All templates are compiled into the binary — no external file dependencies at runtime.

Template Engine (`internal/generator/renderer.go`)

Uses Go's `text/template` with a custom FuncMap:

Function	Description	Example
`lower`	Lowercase	`MyApp` → `myapp`
`upper`	Uppercase	`MyApp` → `MYAPP`
`pascal`	PascalCase	`my_app` → `MyApp`
`camel`	camelCase	`MyApp` → `myApp`
`snake`	snake_case	`MyApp` → `my_app`
`plural`	Pluralize	`item` → `items`
`join`	Join slice	`["a","b"]` → `"a, b"`
`sub`	Subtract	`sub 5 2` → `3`
`last`	Last element	`last ["a","b","c"]` → `"c"`

Template Conventions

- File extension: `.swift.tmpl` → outputs as `.swift`
- Dynamic filenames: `{{.FeatureName}}Model.swift.tmpl`
- Conditional blocks: `{{ if .IncludeSwiftData }}...{{ end }}`
- Template variables via `ProjectTemplateContext` or `PbxprojContext`

Dependency Checker

SystemChecker (`internal/deps/checker.go`)

Runs 5 sequential checks on system startup:

Dependency	Required	Command	Version Parser
Xcode	**Yes**	`xcodebuild -version`	First line of output
git	**Yes**	`git --version`	Extract `git version X.Y.Z`
claude-code	No	`claude --version`	Extract version number
swiftlint	No	`swiftlint version`	Raw output
swiftformat	No	`swiftformat --version`	Raw output

Environment Enrichment

- Adds `/opt/homebrew/bin` and `/usr/local/bin` to PATH
- Falls back to safe working directory (`$HOME` or `/tmp`) if CWD is deleted

DependencyReport

```
type DependencyReport struct {
    Dependencies []Dependency
}

type Dependency struct {
    Name         string
    Required     bool
    Installed    bool
    Version      string
    InstallHint  string    // URL for manual installation
}
```

`Ready()` returns `true` only when all required dependencies are installed.

Error Handling

Custom Error Types (`internal/config/errors.go`)

Error Type	When
`MissingDependencyError`	Required system tool not installed
`TemplateRenderError`	Template parsing or execution failed
`RollbackError`	Cleanup after generation failure also failed
`NoAnvilProjectError`	No `.anvil.yml` found (for `anvil feature`)
`XcodeProjectError`	.xcodeproj generation failed
`FeatureExistsError`	Feature directory already exists
`PackNotFoundError`	Unknown pack slug in selection
`PackDependencyError`	Missing required dependency pack
`SettingsMergeError`	JSON settings merge failed

All implement the `error` interface with descriptive `.Error()` messages and optional `.Unwrap()` for error chaining.

Testing Strategy

Unit Tests

Package	File	Coverage
`config`	`namer_test.go`	Name case conversions (Pascal, Camel, Snake)
`config`	`pack_test.go`	Dependency resolution, pack lookup
`config`	`config_test.go`	Default values, normalization
`deps`	`checker_test.go`	Mock command execution
`deps`	`parser_test.go`	Version string parsing
`generator`	`generator_test.go`	Full generation with mocked writer
`generator`	`renderer_test.go`	Template rendering, FuncMap
`generator`	`xcodeproj_uuid_test.go`	Deterministic UUID generation
`generator`	`xcodeproj_context_test.go`	Pbxproj context building
`generator`	`project_layout_test.go`	File layout with config variations
`feature`	`namer_test.go`	Feature name validation
`feature`	`layout_test.go`	File job generation
`tui`	`layout_test.go`	Layout helpers, version, separator
`tui`	`mode_test.go`	Mode selection navigation
`tui`	`pack_picker_test.go`	Pack toggle, deps, scope prompt
`tui`	`setup_test.go`	Feature tree, expand/collapse, dependency resolution

Integration Tests (build tag: `integration`)

File	Coverage
`integration_test.go`	Full project generation with real templates
`integration_feature_test.go`	Feature scaffolding into generated project
`integration_options_test.go`	All config option combinations

Pattern: Use `t.TempDir()`, mock external commands (Git, Xcode), verify file structure and content.

Golden Tests

File	Coverage
`generator/golden_test.go`	Compare generated output against saved snapshots

CI/CD & Release Process

CI Pipeline (`.github/workflows/ci.yml`)

Trigger: Pull requests to `develop` and `main`

Jobs:

1. **Go Tests** (macos-latest)
   - Checkout → Setup Go → `make test`
2. **Swift Tests** (macos-latest)
   - Install tools (swiftlint, swiftformat, xcbeautify)
   - Build Example project: `xcodebuild build -scheme Arquitectura-Dev`
   - Test Example project: `xcodebuild test -scheme Arquitectura-Dev`

Release Pipeline (`.github/workflows/release.yml`)

Trigger: Tags matching `v*`

Job:

- Checkout with full history → Setup Go → Run goreleaser

Goreleaser Configuration (`.goreleaser.yml`)

```yaml
version: 2
project_name: anvil

builds:
  - id: anvil
    main: ./cmd/anvilcli
    binary: anvil
    env: [CGO_ENABLED=0]
    goos: [darwin]
    goarch: [arm64, amd64]
    ldflags:
      - -s -w
      - -X main.Version={{.Version}}

archives:
  - id: anvil-archive
    format: tar.gz
    name_template: anvil_{{.Version}}_{{.Os}}_{{.Arch}}
    files: [install.sh]

release:
  github:
    owner: magnoscg
    name: Ios-Architecture
  draft: true
  prerelease: auto
```

Release Process

```
git tag v0.3.0
git push origin v0.3.0
# GitHub Actions triggers goreleaser automatically
# Creates draft release with:
#   - anvil_0.3.0_darwin_arm64.tar.gz
```

```
#   - anvil_0.3.0_darwin_amd64.tar.gz
#   - checksums.txt
```

Architecture Decisions

1. Embedded Templates (go:embed)

All templates are compiled into the binary via `go:embed`. This means:

- Zero external file dependencies at runtime
- Single binary distribution
- Templates are versioned with the code

2. Deterministic UUID Generation

Xcode project UUIDs use FNV1a hashing of `projectName + seed`. Benefits:

- Same project name always produces identical UUIDs
- Reproducible project files
- No random state needed

3. Two Generation Modes

Separate `Generate()` and `GenerateToolsOnly()` methods share the same PackRenderer but differ in scope. This enables installing AI tools into existing projects without regenerating the full scaffold.

4. Dependency Resolution Graph

`ResolveDependencies()` uses iterative transitive closure (not recursive) for simplicity. Packs are returned in stable display order. This prevents circular dependency issues and ensures consistent ordering.

5. Marker File (.anvil.yml)

YAML-based project metadata enables:

- `anvil feature` to find project context from any subdirectory
- Version tracking for future migration support
- Backward compatibility via legacy field migration

6. BubbleTea v2 TUI

Modern terminal UI framework chosen for:

- Composable sub-models (one per screen)
- Cross-platform rendering (macOS/Linux)
- Clean separation of update logic and view rendering
- Alt-screen support (clean terminal on exit)

7. Rollback on Failure

Steps 1-6 of project generation support full rollback (directory removal). Steps 7-8 (Xcode project, git) are non-fatal to avoid losing generated code if only the project file or git fails.

8. Pack Composition

CLAUDE.md is composed by layering sections from multiple packs. Each pack can provide either a full `CLAUDE.md.tmpl` (base) or a `CLAUDE-section.md.tmpl` (appended). This enables modular documentation without conflicts.